

主題 : Math Defense

黃晨瑋 41211004E:Full-Stack Developer (Tech Lead · Game Designer)

王晨嫻 41311018E:Frontend Developer (Design Lead · QA)

邵顯智 41411009E:Visual Developer (Game Artist · Video Editing)

GitHub link: <https://github.com/2026-NTNU-Computer-Programming-II-POLS/Math-Defense>

目錄

1. 專題簡介與主題關聯
2. 必要聲明
3. 遊戲玩法
4. 塔、敵人與法術詳解
5. 系統架構
6. 技術實作亮點
7. 創意、新意與原創性
8. 美術與介面設計
9. 安裝與執行方式(Ubuntu 24.04)
10. 開發歷程與遭遇的困難
11. 分工
12. 結論與未來展望
13. 參考資料與素材來源
14. 附錄

1. 專題簡介與主題關聯

塔防(Tower Defense)是一種常見的遊戲類型:敵人沿著固定路徑前進,玩家在沿途建造防禦塔來阻止它們抵達終點。Math Defense 是一款以塔防為骨架的教育遊戲,但它把這個框架作了一個關鍵的翻轉。在絕大多數所謂的「數學遊戲」裡,數學題只是一道門檻,玩家必須先答對題目,才能繼續享受遊戲本身;數學與樂趣是分開的兩件事。Math Defense 的核心設計理念恰好相反,這款遊戲主張數學就是遊戲機制本身,而不是進入遊戲的門票。玩家所選擇的函數、掃過的弧度、計算的向量內積與極限,本身就構成了攻擊。數學即機制,是貫穿這款遊戲的第一主軸;第二主軸則藏於隱蔽處:一款

帶有排行榜與班級競賽的教育遊戲,它的分數必須值得相信。本報告將沿著這兩條軸線展開。

具體來說,整個遊戲世界是一個座標平面。每一個關卡都會在平面上放置一個玩家必須防守的目標點,這個點並非任意指定,而是該關卡中所有敵人行進路徑曲線的唯一共同交點。換句話說,系統會先生成數條都通過同一點的多項式曲線作為敵人的進攻路徑,而它們共同的交會處,正是玩家要守住的據點。敵人從函數與座標平面邊緣的交點出發,沿這些曲線朝目標前進,玩家則建造各自對應到不同數學主題的「塔」來攔截它們,能否命中敵人、造成多少傷害,取決於玩家如何設定它背後的數學運算,例如掃描塔的扇形涵蓋範圍、矩陣塔配對向量的內積大小、極限塔選答案的數學意義等等。

就主題關聯性而言,Math Defense 是一款可以完整遊玩的電腦遊戲,具備關卡選擇、即時戰鬥、計分、成就與天賦進程、排行榜,以及班級之間的領土競賽,與「遊戲」這個題目的關聯明確而直接。

在技術層面,這款遊戲採用三層鬆耦合的執行環境:以 Vue 3 與 TypeScript 撰寫的前端遊戲引擎、以 FastAPI 與 PostgreSQL 構成的後端服務,以及以 C 語言撰寫、再編譯成 WebAssembly 的數學運算核心。其中計分公式的核心與關卡生成完全由 C 語言實作,後端能載入同一份二進位檔重新驗算,藉此防止玩家竄改分數。這個「同一份數學邏輯在前端遊玩、在後端重算驗證」的設計,是本專案在工程上最具代表性的特色,後續章節會詳細說明。

2. 必要聲明

2.1 一魚兩吃聲明

本專案無一魚兩吃，亦非延續任何在本學期開始前即已完成的專案。本專案自本學期課程開始後從零開發，開發過程可由版本控制紀錄完整佐證。

2.2 LLM 使用聲明

本專案在開發全程使用了大型語言模型 (LLM) 程式輔助工具。主要工具為 Claude Code，以代理形式直接參與程式撰寫、除錯與重構，貫穿整個開發期間；另曾輔助使用 ChatGPT 與 Gemini，用途包括美術設計的發想、個別錯誤的除錯討論，以及對程式碼與文件的稽核複查，其中部分除錯產出的程式碼經組員確認後也進入了程式碼庫。為符合課程的揭露要求，以下據實說明使用的範圍與方式。

就程式碼撰寫分布而言，整體約七成由 LLM 撰寫、三成由組員撰寫。需先聲明，兩者在許多模組內部是交錯的，並非以模組為單位整塊切分，故採比重描述如下。LLM 撰寫比重最高的部分，是後端服務、C 數學核心，以及前端遊戲引擎中戰鬥、移動、回放等系統層；組員親手撰寫比重較高的部分，則是前端各主要畫面的版面與互動、設計系統的色彩與元件樣式，以及法術特效與敵人視覺的繪製與調校。在程式碼之外，如遊戲玩法設計、系統架構決策、各項數值的調校與校準等所有取舍判斷，皆由團隊主導；LLM 扮演的是依規格進行實作的角色。專案文件方面同樣有 LLM 參與協助分析和撰寫。

所使用的 prompt 多為功能規格描述，會說明功能的預期行為與邊界條件，指出必須遵守的既有規範，例如決定論、回放相容性與分層邊界，並在產出後反覆迭代修正；除錯時則會附上重現步驟與相關的錯誤訊息。

本專案建立一整套可驗證的機制來確保程式行為正確。這些機制包括持續整合 (CI) 中的多道靜態檢查、確定性回放測試、前後端 WebAssembly 數值對拍 (以同一組輸入比對兩端實作的輸出是否一

致)、後端針對真實資料庫的測試、資料庫層的不變式約束，以及伺服器端的分數重算。這些機制實際存在且可現場展示，第六章會逐一說明。凡是 LLM 協助產出的程式碼，一律與人工撰寫的程式碼通過相同的審閱與 CI 關卡。

2.3 原創性聲明

本專案的核心為「以數學函數塔防為主體、並配合可驗證決定論計分」的遊戲設計與系統架構，皆為本組原創，絕無延伸自任何前人的既有專案。塔防作為一種遊戲類型，「敵人沿路徑前進、玩家建塔攔截」的基本慣例屬於公共的遊戲語彙；但本作的具體機制，包括把防守據點定義為所有路徑曲線的唯一共同交點、開局親手解聯立方程式的初始作答、七種以數學運算本身作為攻擊依據的塔，以及伺服器端可驗證的計分，皆為本組自行設計，沒有對應的參考原型。完整的遊戲初版設計規格詳見專案內的文件所示，開發歷程可由版本紀錄佐證。

於實作層面，本專案沒有參考任何其他遊戲的原始碼，亦無使用 Unity、Godot、Phaser 之類的現成遊戲引擎或遊戲框架。遊戲迴圈、ECS 風格的系統架構、Canvas 渲染、音效合成、錄製與回放，都是自行搭建的。這一點可以由相依清單直接驗證：打開 `frontend/package.json`，執行期相依 (dependencies) 僅有四個通用開源函式庫，即 Vue、Pinia、Vue Router 與數學排版用的 KaTeX，沒有任何一個是遊戲引擎或遊戲機制相關的函式庫，其餘項目皆為建置與測試用的開發工具；換句話說，畫面上看到的每一發砲彈、每一條曲線、每一個音效，都沒有現成的函式庫在背後代勞。當然，「從零搭建」指的是遊戲層而非一切：本專案仍然站在公開的基礎設施之上，包括前後端的開源框架與函式庫、Emscripten 與 musl 工具鏈，以及公開文獻中的 PCG 偽隨機數演算法。原創的是遊戲本身與其上的所有機制，而非底層工具，兩者的清單見第十三章。

另需揭露，遊戲中的班級領土競賽玩法，在「由教師建立限時活動、學生在地圖上搶占格子、

活動結束後依排名結算」此概念層面參考了葉丙成教授團隊的 PaGamO。本專案與其的根本差異在於，Math Defense 的遊戲核心是數學函數塔防加上伺服器端可信重算的計分，搶地的勝負由這套可被重新驗算的分數決定，而非單純的答題正確率；領土競賽僅為作用於此遊戲核心上的一層班級競爭機制。

2.4 素材來源聲明

程式碼由本組成員與 LLM 工具共同完成，其分布與品質把關機制如 2.2 節所述。第三方函式庫皆為開源套件，依各自的授權條款使用，清單見第十三章。影像、字型與音效的來源與授權亦詳見第十三章。本專案整體以 Apache License 2.0 釋出。

3. 遊戲玩法

3.1 一場遊戲的流程

玩家從主選單進入，先選擇關卡的難度星等，範圍從一星到五星。選定後，遊戲不會立刻開始，而是先進入一個稱作「初始作答」的環節，接著才反覆地在「建造階段」與「進攻波次」之間切換，直到所有波次結束後進行結算。全流程的設計特意把思考與動手的時間和真正計分的戰鬥時間分開。

初始作答是此遊戲的特色開場。每一關開始前，系統會以分數的形式列出本關所有敵人路徑的方程式。前面提過，這些曲線剛好只有一個共同交點，也就是玩家要防守的目標據點。為了把搜尋範圍縮小到合理的程度，系統會另外提供一個保證包含該交點的數值矩形範圍，標示出交點的 x 與 y 各自的上下界。玩家必須親手解這組聯立方程式，算出交點的精確座標，並以分數、整數或精確小數的形式輸入，例如二分之三或負四分之五。作答的結果有四種情況。答對的話，計分公式中的一個指數會變得對玩家更有利，使最終分數更高，而且遊戲中會正常顯示敵人路徑。答錯或選擇花五十枚金幣跳過，則該指數不會獲得此加成，但路徑仍然會顯示出來。第四種選擇最為大膽，玩家可以直接開始而完全不顯示敵人路徑，在看不見路線的情況下盲

打，難度最高，適合追求挑戰的玩家。為讓此挑戰名副其實，選擇盲打之後，介面不會留下任何可以反推路徑位置的線索：棋盤上不再標示哪些格子因為臨近路徑而禁止建塔，放塔游標不再以顏色提示該處能否放置，鍵盤操作的游標也可以走到任何格點。這些格子實際上仍然不能放塔，只是玩家無法用看的得知，只能實際嘗試。另需說明，盲打並沒有額外的分數補償，初始作答的指數加成同樣不會生效。

建造階段是沒有敵人的準備時間。玩家在這個階段放置、升級或退費自己的塔，設定每一座塔背後的數學參數，例如要畫哪一條曲線、扇形掃描的弧度、向量如何配對、極限題選哪個答案、要對哪個多項式做微分或積分，亦可設定每座塔鎖定目標的策略或在商店購買增益效果。值得一提的是，停留在建造階段的時間並不會被計入積分所用的有效作戰時間，故慢慢思考佈陣並不會被扣分，但系統不因玩家長時間掛機在建造階段而給予額外好處。

進攻波次則是真正的戰鬥。塔會依照設定自動開火，其設定在波次進行中將無法被更動，但玩家仍可施放法術、暫停，或加速與減速遊戲進程。一關通常包含三到五個波次，星等越高，波次越多，敵人組合也越凶險。

3.2 七種對應數學主題的塔

此遊戲共有七種塔，各對應一個數學主題，而該主題的運算就是攻擊的依據：魔法塔把玩家選定的曲線畫成帶狀的作用區域，三種雷達塔（掃描、速射、狙擊）建立在弧度與扇形的概念上，矩陣塔以配對向量的內積決定傷害，極限塔由極限選擇題的答案決定效果，微積分塔則依玩家算出的微分或積分結果生成寵物。為讓新手不會一開始就被進階數學難住，塔沿著關卡難度分批解鎖；每座塔的造價、數值、運作機制與升級細節，於第四章逐一詳述。

3.3 敵人與特殊事件

遊戲共有十種敵人，各自有相剋的玩法，構成戰術深度：從作為基準的一般兵，到快速、強壯、分裂、輔助、再生等特化兵種，再到帶高傷害抗性

的堡壘兵與蟲群、兩種行為迥異的頭目，迫使玩家隨敵人組合調整佈陣；各敵人特性介紹詳見 4.2 節。

遊戲過程中可能觸發帶有教學意圖得的兩種特殊事件。第一種是蒙提霍爾事件，當累積擊殺價值達門檻時，波次會暫停，玩家先從三到五扇門中選定一扇(門數隨星等增加)，系統接著把除了玩家所選與另外一扇之外的必輸之門全部揭開，再讓玩家決定要維持原選擇，還是換到僅存的那一扇。在數學上，更換的勝率是門數減一再除以門數，三扇門時是三分之二，五扇門時是五分之四，但遊戲不會告知這個事實，讓玩家從反覆遊玩中自行體會「換門比較有利」這個違反直覺的條件機率結論。第二種是連鎖律挑戰，當 B 型頭目血量降到約一半時出一道題，答對則可以秒殺頭目並獲得一百枚金幣，頭目隨即碎裂成兩隻已失去護盾的小敵；答錯則不扣血也不懲罰，頭目回到戰鬥，遊戲繼續進行。

3.4 計分公式

分數核心由 C 實作 (wasm/math_engine.c)，因此後端能以位元等級的精確度重新驗算，前端只負責把同樣的結果鏡像顯示給玩家看。計分的具體形式如下，其中各項變數的意義會在公式之後說明。

```
activeTime =
    max(0.001, timeTotal - Σ 建造階段耗時)
S1          = killValue / activeTime
S2          = killValue / costTotal
              (costTotal 大於 0，否則為 0)
alpha       = S1 / (S1 + S2)
              (S1 + S2 大於 0，否則為 0)
K           = alpha·S1 + (1 - alpha)·S2
exponent    = 1 /
              sqrt(
                  max(1,
                      1 + (2 + healthOrigin - healthFinal
                        - initialAnswer)
                  )
              )
core         = max(0, killValue)^exponent · K
TotalScore  = core · SCALE · difficulty
```

公式的設計目標是同時獎勵「又快又省地擊殺敵人」與「把學習成效反映到分數」。分數的量體基礎是擊殺價值，擊殺得越多，分數的基底越大。其中 K 是兩種效率的連續混合：S1 代表單位時間的擊殺效率，S2 代表單位成本的擊殺效率，兩者依各

自占比平滑地加權，避免早期版本用固定比例折線所造成的分數跳變。指數部分則把學習成效收進來：玩家若在初始作答答對，以及在整局中少掉血，這個指數就會偏大，分數隨之提高；反之掉血越多、開局題答錯，指數越小，分數被壓低，但因為用了平方根，懲罰是漸進的而非斷崖式，仍保留鼓勵性。

另需說明計分的兩段式結構。公式的 core 部分，也就是擊殺價值的指數次方再乘上效率混合 K，完全在 C 語言核心內計算；前端顯示與後端重算執行的是同一份 WebAssembly 二進位檔，輸出逐位元一致，而 C、Python、TypeScript 三方實作另以共用的對拍資料鎖定在十的負十二次方的容差之內，僅容許不同數學函式庫在最末位上的差異。至於量體縮放 SCALE 與星級難度乘數 difficulty，則於伺服器端由可信的場次資料另外套用，不在 C 核心內。其中 SCALE 固定為一，僅為預留的調節旋鈕；difficulty 等於一加上零點二五乘以星等減一，故一星為一倍、五星為兩倍，用來獎勵挑戰高星等關卡。

用一個例子把公式走一遍。假設一局三星遊戲的擊殺價值為六百，有效作戰時間一百二十秒，建塔總成本四百金幣，玩家答對了初始作答，而且整局只掉了兩點生命。此時 S1 為每秒五分、S2 為每金幣一點五分，混合出約四點一九的效率值 K；指數的內層為 $1 + (2 + 2 - 1) = 4$ ，開根號取倒數後得到指數零點五，核心分數因此是六百的零點五次方乘以 K，約一百零三分，再乘上三星的一點五倍難度，總分約一百五十四分。若同一局完全沒掉血，指數會升到約零點七一，總分則躍升到約五百八十分。掉血對分數的影響漸進而顯著，這正是公式想傳達的訊息：防守品質本身就是學習成效的一部分。

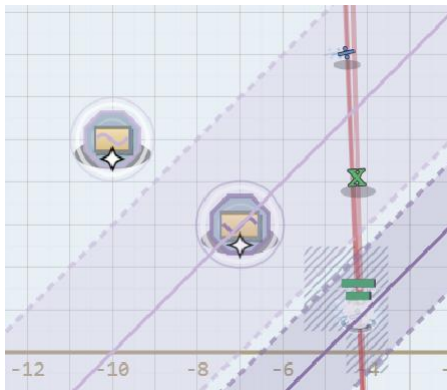
4. 塔、敵人與法術詳解

本章將把各塔、敵人與法術逐一展開，介紹每一項背後的數學概念與運作機制。塔的示範影片一律依「建置與設定、遊玩、更新設置、再次遊玩」的順序錄製，各塔互相獨立、各有一個連結。

4.1 七種塔

第三章已概覽過七種塔的定位，本節補上每一座的造價、數值與升級細節。解鎖節奏沿著難度階梯安排：關卡難度第一星先開放魔法塔與掃描雷達塔，第二星追加速射雷達塔、狙擊雷達塔與矩陣塔，第三星再解鎖牽涉高三內容的極限塔與微積分塔。每一座塔都另有第二與第三階升級，通用效果為第二階增加兩成五傷害與一成射程、第三階增加五成傷害與兩成射程並附帶一成五攻擊速度，各塔特有強化則於下方分別說明。其中三種雷達塔(掃描、速射、狙擊)都可在面板上設定一段攻擊弧度，預設為全圓掃描、扇形僅提供傷害乘以一點五倍的聚焦加成，開啟限制後則只攻擊落在扇形內的敵人。

魔法塔(✦)

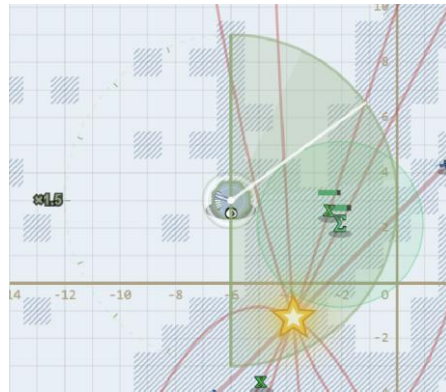


<https://drive.google.com/file/d/1oADg-DH5oPRHCvxNQCghy3n1-1VOite/view?usp=sharing>

魔法塔對應的數學主題是函數曲線，玩家可以從多項式、三角函數與對數函數三族中挑選一條，系統會把這條曲線畫成一條帶狀的作用區域。它在關卡難度第一星即可解鎖，造價六十金幣，基礎傷害八點，射程十格，冷卻一秒。此塔有兩種可切換的模式：在減益模式下，落在帶狀區域內的敵人會持續受到傷害，並被減速到原速的六成、效果持續兩秒，且每次再被命中都會刷新；在增益模式下，落在同一條曲線(較寬的帶)內的友方塔則會獲得傷害加成。由於兩秒的減速長於一秒的傷害節拍，單獨一座魔法塔就能讓敵人在兩次傷害之間維持連續減速。實戰上的訣竅是讓所選曲線盡量貼合敵人即將行經的路徑，若要作輔助用途，則讓曲線穿過自己

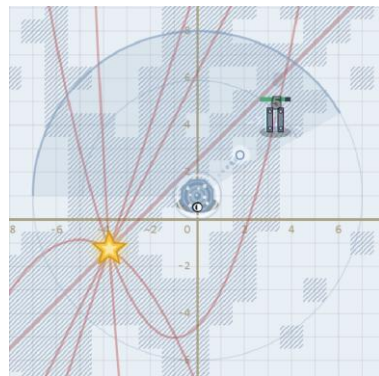
的塔群。這座塔所練到的正是學測數學 A 的多項式與三角函數圖形與 AP 先修微積分的內容。

掃描雷達塔(📡) & 速射雷達塔(📡) & 狙擊雷達塔(📡)



https://drive.google.com/file/d/1fWR3L09LHy3WLmulUx_53ik3UzUtKLaz/view?usp=sharing

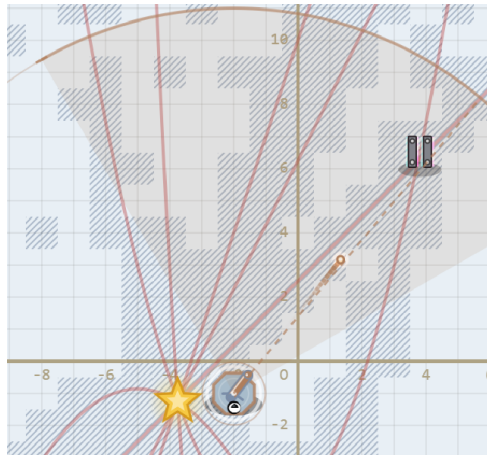
掃描雷達塔對應弧度與扇形面積的概念，同樣在關卡難度第一星解鎖，造價五十金幣，基礎傷害五點，射程六格，冷卻半秒。它以一根繞塔旋轉的指針掃描，指針每掃過一個敵人就造成一次傷害，屬於一次乾淨的命中而非持續的滴傷；由於指針會掃過範圍內的每一個敵人，它適合對付擠成一團的低血量敵群，代價是單一敵人承受的傷害刻意壓得很低。指針掃得越快、單位時間掃過的次數越多，實際輸出也越高。預設指針會繞行完整的三百六十度，玩家設定的扇形只是標出一塊傷害乘以一點五倍的聚焦區；若開啟限制攻擊弧度，指針就會變成只在該扇形內來回擺動的兩刷，忽略扇形外的目標。它的特化升級是把掃描速度提高兩成與四成，由於傷害來自掃過的次數，提速等同直接增傷；第三階另外把偵測帶加寬零點三弧度，使攔截範圍再擴大。



<https://drive.google.com/file/d/1CBs3u2491fZDARSqMaDajWw35inUhgXX/view?usp=sharing>

速射雷達塔在關卡難度第二星解鎖，對應的數學概念仍是弧度。它造價六十五金幣，基礎傷害八點，射程七格，冷卻零點三秒，是冷卻最短的一

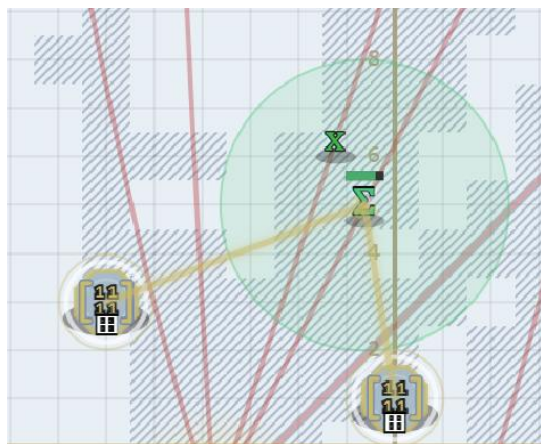
座，專應付密集而快速的目標。預設朝所有方向開火，設定的扇形可作為傷害乘以一點五倍的聚焦區，或在開啟限制後只攻擊扇形內的敵人。它的特化升級是讓它能同時鎖定多一個與多兩個目標，使它在面對成串的小型快速敵人時，火力更為分散而綿密。



https://drive.google.com/file/d/1nUP0CGX5W_hlIqNf5-cwUth801tZkZ6k/view?usp=sharing

狙擊雷達塔同在關卡難度第二星解鎖，概念一樣建立在弧度上。它造價九十金幣，基礎傷害高達四十點，射程十二格，但冷卻長達兩秒半，是全部塔中射程最遠、單發傷害最重的一座，適合處理高血量目標。它的彈道會預判並提前瞄準移動中的敵人，設定的扇形提供傷害乘以一點五倍的聚焦，並可選擇是否限制只打扇形內。它的特化升級是提高一成與兩成的暴擊機率，第三階再追加五成的暴擊傷害，讓每一發都更具決定性。

矩陣塔(冏)

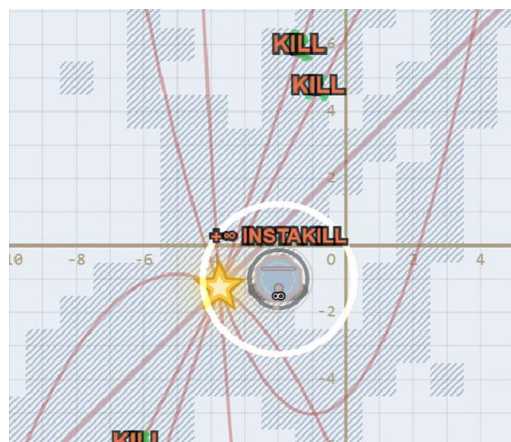


https://drive.google.com/file/d/1L_ZBjvyEHW6j825Zx-9q-2CNLPkFKPzS/view?usp=sharing

矩陣塔對應向量與內積，在關卡難度第二星解鎖，造價八十金幣，射程八格，冷卻半秒。它最特別之處在於必須成對運作，單獨一座完全不會開

火；玩家透過面板把兩座矩陣塔配成一對後，這對塔的基礎傷害等於一加上兩座塔各自格座標所構成向量的內積。它以雷射鎖定同時落在兩座塔射程交集內的目標，鎖得越久傷害會逐步攀升，一旦目標離開交集或被擊殺，鎖定就解除、累積的加成歸零。它的特化升級是把鎖定增傷的速率提高兩成與四成，第三階更讓雷射能多掃一個目標。它適合在能擺出有利內積的位置、且場上有單一高血量的目標(例如小頭目、堡壘兵或頭目)時使用。這座塔練的是高二數學的向量內積，以及高三選修數學甲的二階矩陣。

極限塔(∞)

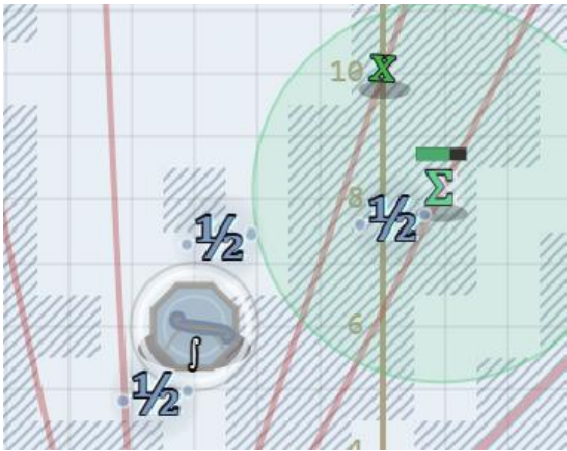


<https://drive.google.com/file/d/1fbZ5-jBf2MH4uv-AX6F7EGptlQMfXZc/view?usp=sharing>

極限塔對應極限的概念，並牽涉羅必達法則，在關卡難度第三星解鎖，造價七十金幣，基礎傷害二十五點，射程八格。它每蓄力三秒便釋放一次範圍爆發，以一點五倍的公式傷害打中射程內的所有敵人。面板會出一道單邊極限的選擇題，形式為 x 從 a 的右側趨近時， $f(x)$ 除以 $(x$ 減 $a)$ 的極限，玩家所選答案的數學意義直接決定效果:答案為正無限大時，範圍內所有敵人會被直接秒殺，且無視堡壘兵與蟲群那類防禦減免；答案為某個有限正數時，傷害會依該數的大小放大，再乘上前述一點五倍的爆發倍率；若答案是零、不存在、負數或負無限大，則只造成削減後的少量傷害。它最適合用在玩家能快速且穩定解出題目、又面對長壽高血量敵人的場合，因為一個正確的正無限大結果有足夠時間扭轉整個波次。答錯或答出退化的答案時，只會變成微量傷害，但在三秒一次的節奏下，失去一次爆發的

代價仍然不小。這座塔對應 AP 微積分 AB 的單邊極限與無窮極限，以及高三選修數學甲的極限單元。

微積分塔(J)



微積分塔對應微分與積分，核心是冪次法則，在關卡難度第三星解鎖，造價一百金幣，射程十格，本身並不直接開火。玩家先挑一個單項式 $f(x)$ ，再選擇一種運算，可以是一階導數、二階導數或積分，而且必須自己動手算出結果並輸入對應的單項式；面板從不顯示答案，只有算對才會套用這次運算。運算結果的形式為 C 乘以 x 的 n 次方，它會生成數隻會自動朝最近敵人追擊並引爆的寵物。其中指數 n 決定寵物的特性： n 等於一是慢速型、等於二是快速型、等於三是重型、大於等於四則是基本型，各自對應不同的單體傷害、攻擊頻率與移動速度，例如快速型打得輕但出手頻繁，重型打得重但較慢。係數 C 則經過對數壓縮換算成寵物數量，實際生成的隻數為以二為底、 C 加一取對數後再向下取整，因此一個九十九乘以 x 平方的結果會生成六隻快速寵物，而非九十九隻；天賦可以再增加數量，非整數的係數絕對值則轉為每隻寵物的傷害倍率。第一次運算免費，之後再串接運算要花金幣，算錯則不收費並直接退回。它的特化升級是把寵物傷害提高兩成五與五成，第三階再多生一隻寵物並提升兩成寵物速度。值得一提的是，寵物的攻擊是少數能繞過堡壘兵與蟲群傷害減免的來源之一，因此這座塔在面對高抗性敵人時格外關鍵。它對應的是 AP 微積分 AB 的多項式微分與積分。

4.2 十種敵人

十種敵人各自針對不同的防守習慣設計，迫使玩家隨敵人組合調整佈陣。以下每一種都標注其生命值、移動速度、擊殺金幣、抵達據點造成的傷害，以及擊殺價值；其中擊殺價值不同於擊殺掉落的金幣，它是用來累積蒙提霍爾事件門檻與計分公式的數值。需要先說明的是，堡壘兵與蟲群對塔傷害帶有高抗性，能繞過減免的來源僅有微積分塔的寵物，以及極限塔正無限大不走傷害流程的瞬殺；玩家施放的法術同樣會被減免。

一般兵 & 快速兵 & 強壯兵



General



Fast



Strong

一般兵是最基本的敵人，生命三十、移動速度二、擊殺給十五金幣、抵達據點造成一點傷害、擊殺價值十，作為衡量所有敵人的基準；快速兵的移動速度高達四，是一般兵的兩倍，但生命只有十五、相當單薄，擊殺給八金幣、傷害一、擊殺價值五，威脅在於速度，需以高射速或減速手段攔截；強壯兵生命厚達一百二十、移動速度只有一，擊殺給三十八金幣、傷害二、擊殺價值二十五，是典型的肉盾型敵人，需要持續而集中的火力才能放倒。

分裂兵 & 輔助兵



Split



Helper

分裂兵生命四十、速度二、擊殺給八金幣、傷害一、擊殺價值五，死亡時會分裂成兩隻縮小到四成的一般兵，宜以範圍攻擊在其預計死亡的位置一併清掉小兵；輔助兵生命三十五、速度二、擊殺給二十三金幣、傷害一、擊殺價值十五，其光環持續為半徑三格內的同伴每秒回復五點生命並提升兩成移動速度，建議優先擊殺。

再生兵 & 堡壘兵



Regenerator

再生兵生命八十、速度一點五、擊殺給三十金幣、傷害二、擊殺價值二十，每秒回復十八點生命，須以狙擊雷達塔或脈衝法術等爆發傷害一口氣打死；堡壘兵生命高達二百二十、速度零點九、擊殺給四十五金幣、傷害三、擊殺價值三十，只承受塔傷害的四成，是場上抗性最高的一般敵人，矩陣塔的雷射可靠持續累積的增傷慢慢磨穿它。



Bulwark

蟲群



Swarming

蟲群生命僅十二、速度三點二、擊殺給六金幣、傷害一、擊殺價值四，僅受塔傷的三成五且成群密集出現，搭配漸近線法術把整群減速較好處理。

A 型頭目



Boss Type-A

A 型頭目生命五百、速度零點八、擊殺給一百五十金幣、擊殺價值一百，接觸據點的傷害高達九十九，是刻意設計的致命值，一旦抵達便直接終結整局。它帶有兩百點護盾，護盾未破前所有塔傷害都打在護盾上而非本體，且每八秒召喚一隻一般兵持續施壓，必須及早以穩定的高輸出處理掉。

B 型頭目



Boss Type-B

B 型頭目生命六百、速度零點七、擊殺給二百二十五金幣、擊殺價值一百五十，接觸傷害同樣是九十九，並帶有兩百五十點護盾、每八秒召喚一隻快速兵。它最大的特色是連鎖律挑戰(觸發與獎懲見 3.3 節):頭目碎裂出的兩隻小敵，一隻是血量為原頭目六成的強壯型、一隻是四成的快速型，皆已被剝除護盾，以一般傷害擊殺它時同樣會分裂；為避免被記憶，觸發的血量比例會在每次生成時於四成五到五成五之間取樣。

以上著重數值與相剋要點，完整說明請見遊戲手冊。

4.3 四種法術

遊戲提供四種可在任何時候施放的法術，它們花費金幣、即時生效、各有冷卻，且與建造階段商店的增益不共用資源，命名皆呼應其數學意涵。

指數法術(e^x) & 漸近線法術($\rightarrow 0$)

指數法術以 e 的 x 次方為名，花費八十金幣冷卻十二秒，在點擊處半徑三格的範圍內造成六十點傷害；它的命名取自指數函數爆炸性成長的意象，適合清掉蟲群的密集波次與分裂兵留下的小兵。

漸近線法術以趨近於零為名，花費六十金幣，冷卻十五秒，讓半徑四格內的敵人減速到原速的四成、持續五秒；命名取自速度被漸近地壓向零的概念，常用來在輔助兵帶頭的波次開場時拖慢全場，好讓玩家先擊殺輔助兵。

脈衝法術(δ) & 加速法術(dv/dt)

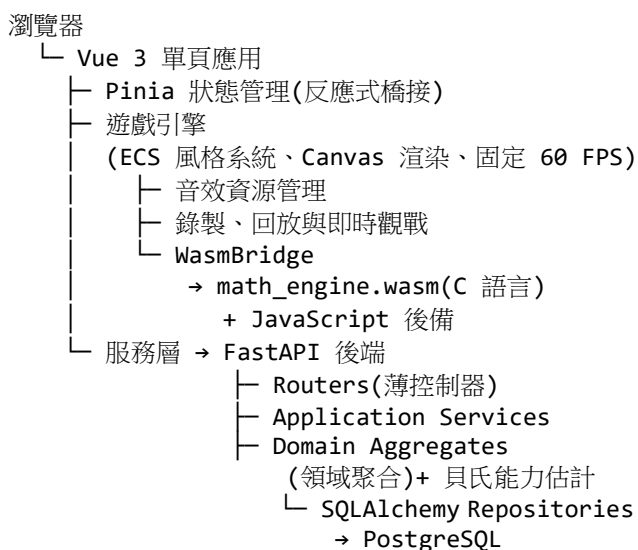
脈衝法術以狄拉克 δ 函數為名，花費一百金幣，冷卻十八秒，對單一敵人造成一百五十點傷害；命名取自 δ 函數把能量集中在一點的特性，適合用來補刀已被打穿護盾的頭目或一擊處理再生兵。

加速法術以速度對時間的導數 dv/dt 為名，花費一百二十金幣，冷卻二十五秒，施放後的八秒內令所有塔造成一點五倍傷害；命名取自輸出如加速度般陡升的意象，作為一種應急用法術。

5. 系統架構

5.1 三層執行環境

Math Defense 由三個鬆耦合的執行層構成，彼此之間透過明確的介面溝通，任一層都不直接依賴另一層的內部細節。三者的關係如下。



前端負責所有的遊戲邏輯、畫面渲染與玩家互動。遊戲的核心進行，包括關卡生成、遊戲迴圈與即時計分，完全在前端執行，而與後端的同步也設計成非阻塞，因此即使同步暫時失敗，單機遊玩的流暢度也不受影響。需要說明的是，由於遊玩頁面受登入保護、而登入仰賴後端，本專案並未提供訪客或離線模式，初次進入仍需後端可連線以完成登入。嵌在前端內的 C 與 WebAssembly 數學核心，處理的是數值密集且需要可重現性的運算，主要是關卡生成與計分。WebAssembly 是一種可以在瀏覽器裡以接近原生速度執行的低階格式，本專案把 C 程式編譯成這種格式來跑這些計算。後端則負責帳號、排行榜、班級與領土競賽、成就與天賦、回放驗證等需要持久化保存與防作弊的部分。

5.2 技術棧

前端以 Vue 3.5 的組合式 API 搭配嚴格模式的 TypeScript 6.0 撰寫，狀態管理使用 Pinia 3，路由使用 Vue Router 5，建置工具為 Vite 8，測試框架為 Vitest 4。後端以 FastAPI 為主，搭配 Uvicorn、SQLAlchemy 2.0、Pydantic v2，認證使用 PyJWT 與 bcrypt，限流使用 slowapi。數學核心以 C99 撰寫，透過 Emscripten 以開啟最佳化並關閉非確定性浮點重寫的旗標編譯，對外匯出十七個數學函式，另外加上記憶體管理用的兩個函式。資料庫為 PostgreSQL 16，結構變更以 Alembic 管理，目前累積近五十個遷移檔、二十九張資料表。整個系統以 Docker 與 Docker Compose 容器化。回放方面採版本化設計，後端以 wasmtime 載入與前端相同的那一份 WebAssembly 二進位檔來重算分數。

5.3 共用常數

為預防前後端的參數於開發過程中各自漂移，本專案採以 shared/game-constants.json 作為單一真實來源，前後端從這份檔案讀取共用數值。其中包含畫布尺寸為一千兩百八十乘七百二十像素、座標格線範圍在正負十四之間、玩家初始為二十點生命值與兩百枚金幣(隨星等最高提高到三百二十枚)、目標每秒六十幀，以及各星等的經濟參數與波次獎勵等。把這些常數集中在一處，確保前端顯示、後端驗算與測試三方對同一組數字有一致的認知。

6. 技術實作亮點

6.1 為什麼使用 C 與 WebAssembly

此專案把 C 語言落實在對數值精確度與可重現性要求最高的數學核心。wasm/ 目錄下以 C99 撰寫了四個原始檔，最後連結成單一的數學引擎二進位檔。math_engine.c 提供二乘二矩陣相乘、扇形面積與「點是否落在扇形內」的判定、梯形法數值積分，以及計分公式等數學函式，其中計分公式是實際在執行期由前端顯示與後端重算共用的關鍵函式。curve.c 處理多項式、三角與對數三族曲線的求值、微分與定義域判斷。level_gen.c 負責關卡

生成，以一個拒絕取樣迴圈反覆嘗試，最多八批、每批五十次，每一個候選關卡都必須同時通過四道檢驗：曲線之間斜率分離得夠開、每條曲線在格線邊界恰好有兩個出生點、揭露範圍唯一，以及目標點確實落在範圍之內。其中求交點用的是變號掃描加二分逼近，而為了讓不同執行環境跑出完全一致的結果，二分逼近的迭代次數一律在進入迴圈前就決定好：交點求解的圈數以對數公式先行算定，出生點求解則採固定圈數，使各執行環境跑完全相同的迴圈次數；曲線的自由係數則一律限制在四分之一的整數倍這種二進位分數格點上，讓生成的曲線能精確通過目標點而不帶浮點誤差。`prng.c` 則是一個 PCG 系列的偽隨機數產生器，供版本二的回放使用。

選擇 C 與 WebAssembly 的理由，重點在於把對精確度與可重現性最敏感的運算集中到一個跨平台行為一致的核心。計分與關卡生成牽涉大量浮點與數值求根，而它們的結果又必須能在後端被位元等級地重算以防作弊，這一點下一節會詳述；把這些放進編譯為 WebAssembly 的 C，既能得到接近原生的執行速度，又能讓同一份二進位檔在前端與後端產生完全一致的輸出，同時讓數學運算層與畫面、文件物件模型徹底解耦。前端永遠透過一個稱為 WasmBridge 的橋接層來呼叫這些函式，從不直接引用 Emscripten 產生的膠合程式碼。

6.2 可驗證的決定論與伺服器端防作弊

這是本專案在工程上最核心的亮點。所謂決定論，指的是同樣的輸入一定產生完全相同的輸出，不帶任何隨機性。本專案追求的是更嚴格的一種：同一局遊戲，不論在哪一種 WebAssembly 執行環境上跑，都要產生位元等級完全相同的浮點輸出。要做到這件事，需要幾個條件同時成立。Emscripten 在編譯時把 `musl` 這套 C 標準函式庫一起編進二進位檔，因此 `sinf`、`cosf`、`logf`、`pow` 這些超越函數是以 WebAssembly 位元碼的形式隨二進位檔附帶，而不是借用執行環境主機內建的版本；執行環境只是逐位元解讀這份位元碼，不會替換成自己的

實作。同時，編譯旗標明確禁止編譯器對浮點運算式做任何會造成微小末位誤差的代數重寫，例如把連乘加融合成單一的乘加指令。亂數產生器本身也為此而設計，它取五十三個確定的隨機位元組成一個雙精度浮點數，對應到可被精確表示的數值，使兩個執行環境即使各自運算也不會因為捨入而分歧。

決定論帶來的直接好處，是後端可以充當一個誠實的裁判。整場遊戲可以由一個亂數種子完整重現，後端以 `wasmtime` 載入與前端完全相同的那一份數學引擎，從玩家上傳的輸入紀錄重算一次分數。對於採用版本二回放的場次，只要玩家提交的分數與伺服器重算的結果差異超過容差，後端就以 HTTP 422 的 `replay_mismatch` 拒絕這筆提交；在其餘情況下後端則一律以伺服器自己重算的值為準。如此一來，玩家無法靠竄改前端來偽造分數，因為最終採信的永遠是伺服器用同一份邏輯算出來的結果。防作弊不只靠重算這一步：伺服器另外從回放事件紀錄推回真正的作戰時間，使玩家無法靠竄改計時器來灌高效率分；當有人宣稱有擊殺卻完全沒有付出建塔成本時，分數直接歸零；而若一個版本二的場次送達、伺服器端的 WebAssembly 卻尚未就緒，系統選擇直接拒絕該筆提交，而非姑且相信前端。最終分數另設有一百萬分的上限，避免溢位與極端值破壞排行榜。回放事件的上傳本身也有防竄改設計，伺服器只接受序號嚴格遞增的新事件，且只允許仍在進行中的場次追加，藉此擋下以舊序號交錯插入、或對已結束場次補寫紀錄的手法。為了確保 C、Python、TypeScript 三套實作不會悄悄分歧，專案以一組固定的「輸入對應正確輸出」對拍資料把三方鎖在一起，任何對計分公式的改動，只要造成三方輸出不一致，就會被對拍測試攔下來。

6.3 C 核心的型別選用與介面捨捨

本節把前兩節的決定論目標再往下落一層，檢視它如何化為 C 語言層面的具體決策，涵蓋浮點精度與整數寬度的選用、跨語言介面，以及記憶體

與錯誤處理的約定；這些決策多半不是孤立的偏好，而是從「哪一種正確性必須被保證」反推出來的。

C 語言把數值的精度與寬度都交給開發者明確決定，本專案刻意在同一個核心裡並用兩種浮點精度，對應兩種不同的正確性契約。計分公式全程使用六十四位元的雙精度浮點數，原因在於它的比對對象是 JavaScript 與 Python:這兩種語言的數字在規格上就是 IEEE 754 雙精度，C 端採用同一種表示法，三方實作才有逐位元一致的可能，上一節的對拍測試也才有明確的判準。關卡生成與曲線求值則把所有係數與座標都放在三十二位元的單精度浮點數上，因為這裡的契約不同:回放從頭到尾只走 WebAssembly 這一條路徑，決定論只要求同一份二進位檔在任何引擎上跑出相同結果，並不要求與僅作後備的 JavaScript 實作逐位元相同。既然不需要跨語言逐位元對齊，單精度較小的體積便成為優點，一條曲線的完整描述因此恰好裝進二十四個位元組。

整數則一律取用 C99 標準的 `stdint.h` 標頭所定義的定寬整數型別，不使用寬度隨平台浮動的 `int` 或 `long`。理由在於，C 核心的結構體同時就是前端與 C 之間的傳輸格式:TypeScript 橋接層把欄位逐一寫進 WebAssembly 的線性記憶體，C 端透過同一份標頭檔解讀同一塊記憶體，因此每個欄位的寬度、順序與對齊，必須在三十二位元的 WebAssembly 與日後可能的六十四位元原生編譯目標上完全一致。實務上，這些結構的欄位都恰好是四個位元組寬，編譯器因此沒有機會插入難以察覺的填充位元組，例如關卡生成的完整回傳就是一筆五百五十六個位元組的扁平紀錄。欄位的順序被視為公開介面的一部分，任何調動都等同於回放格式的破壞性變更，必須隨之提升回放的版本號。

亂數產生器最能體現內部表示與對外介面可以分開選型。它的內部狀態是六十四位元的無號整數，這是 PCG 演算法的週期能達到二的六十四次方的前提；但對外的種子與序流參數刻意宣告為三十二位元，因為 Emscripten 的呼叫橋接會把六十四位

元整數映射成 JavaScript 的 `BigInt` 型別，徒增邊界上的轉換負擔與出錯空間，而三十二位元的種子空間已足以涵蓋一場遊戲所需的隨機性。上一節提到亂數輸出以五十三個隨機位元組成一個雙精度浮點數，之所以恰好取五十三，是因為這正是 IEEE 754 雙精度的有效位數:湊滿但不超過這個位數，每個輸出都會落在可被精確表示的格點上，捨入因此沒有發揮的餘地，這是把型別規格本身當作演算法設計依據的具體例子。

記憶體管理採取 C 端零配置的原則。整個核心沒有任何一處呼叫 `malloc`:跨越語言邊界的狀態與緩衝區一律由呼叫端配置，C 函式只透過指標讀寫；凡是輸出筆數不定的函式，都要求呼叫端一併傳入容量上限，並回傳實際寫入的筆數，只要容量如實申報，C 端就不會越界寫入；至於函式內部的暫存空間，則全是編譯期就定下大小的堆疊陣列。這個原則的代價是所有規模上限都必須在編譯期定死，例如一關至多八條曲線、十六個出生點，但這些上限本來就由遊戲設計保證，被犧牲的彈性實際上用不到；換來的好處是，動態記憶體的生命週期全數集中在 TypeScript 橋接層一處，也就是第十章所述以自動釋放的包裝函式統一管理的位置，洩漏若會發生就只能發生在那裡，C 核心本身則在結構上排除了這種錯誤。

錯誤處理同樣有明確的約定。C 沒有例外機制，核心因此統一以回傳值表示成敗、以輸出參數帶回結果，與 TypeScript 端以「數值或空值」表達可能失敗的求值一一對應；對數曲線在定義域外的求值刻意回傳浮點數的非數值(NaN)，讓誤用產生一個一眼可辨的錯誤訊號，而不是回傳零之後無聲地混進後續計算。最後一項取舍，是把格線範圍這類常數直接寫死在 C 原始碼裡，而不是開放成函式參數:這些數值屬於決定論契約的一部分，做成參數等於允許呼叫端各自傳入不同的值，讓同一顆種子生成出不同的關卡；寫死之後，一份二進位檔就只

有一種行為，想改變它們就必須連同回放版本一起升級，而這正是本專案需要的保證。

6.4 後端領域驅動設計與資料庫作為最後防線

後端採領域驅動設計，將「遊戲與系統的規則」和「網路、資料庫這些技術細節」分開存放。核心的業務規則，例如誰能搶領地、分數怎麼算、成就怎麼解，被寫成不綁任何框架的純邏輯，放在領域層；往外依序是協調流程的應用服務層、實際接資料庫的基礎設施層，以及接收網路請求的路由層。領域層不含 HTTP 概念，領域內拋出的錯誤是不帶狀態碼的，真正把它們翻譯成 HTTP 狀態碼這件事，集中在一個對照表裡處理。如此分層讓核心規則可以脫離瀏覽器與伺服器、用純 Python 單獨測試，例如代表一場遊戲的聚合，其測試完全不需要開資料庫就能跑。這套設計之所以划算，是有三個現實需求把它推導出來：分數必須能在伺服器端被可信地重新驗算，業務規則不能只活在前端；系統的子領域確實很多，從遊戲場次、成就、天賦，到領地、班級、賽季，各有各的規則，不分層很快就會纏成一團；把領域與資料庫解耦，日後更換儲存方式或新增功能時影響面才會小。在這套分層之上，一場遊戲結束所衍生的各種副作用，例如更新排行榜、檢查成就、更新初始作答的近期正確率，是透過一個領域事件匯流排分派出去的。處理器各自在獨立的工作單元中執行、各自捕捉例外，因此其中一項失敗不會連帶回滾已經穩定寫入的場次紀錄。

除了在程式層做檢查之外，本專案刻意把真正的不變式進一步下放到資料庫本身，作為最後一道防線。程式會有疏漏，但只要某些事實是「永遠都該成立」的，就把它釘在最底層、最不容易被繞過的地方。舉例而言，資料庫透過一個帶條件的唯一索引，保證同一個使用者最多只能有一場進行中的遊戲，從根本上杜絕了用分身同時開多局來刷分的可能；另有多個檢查約束保證分數、擊殺數、生命值不為負，星等落在一到五之間，作答正確率落在零到一之間等。這類不變式本來就不該改變，放

進資料庫反而是最穩固的做法，而日常會變動的業務流程則仍然留在應用層，維持彈性。排行榜寫入、回放事件與成就解鎖也都採具冪等性的寫入，透過唯一索引或「衝突時不重複插入」的方式達成，並由場次列的鎖把整個結算流程串行化，確保在高併發或重試時不會產生重複或遺失更新。

6.5 認證與安全縱深

在進入個別機制細節前，先以縱深防禦的全景圖總覽整體布局。下圖把後端想像成一條由外而內的走廊：一個不可信的請求從網路進來，必須依序通過八道彼此獨立的防線，才能觸及持久化的資料，其中任何一道被繞過，後面仍有防線接手。不同類型的請求會遭遇其中不同的子集，例如尚未登入的請求不需驗證權杖，卻要面對帳號鎖定與限流。

不可信的用戶端請求

(前提：任何欄位、Cookie 與分數都可能被竄改)

- ▼ ① nginx 邊界：TLS、安全標頭、請求體上限 1 MB
攔下：竊聽、點擊劫持、內嵌腳本、超大請求
- ▼ ② 來源與流量：CORS 白名單、逐端點每 IP 限流
攔下：陌生網域的跨來源呼叫、洪水式請求
- ▼ ③ CSRF 防護：雙重提交 Cookie、常數時間比對
攔下：跨站請求偽造、時序探測
- ▼ ④ 權杖驗證：短效 JWT、登出黑名單、密碼版本比對、更新權杖旋轉與家族撤銷
攔下：演算法混淆、舊權杖重用、更新權杖重放
- ▼ ⑤ 輸入驗證：Pydantic+領域複驗、圖片魔術數字
攔下：未知欄位、偽裝圖片、解壓縮炸彈
- ▼ ⑥ 帳號防線：升級式鎖定、假雜湊比對、TOTP 加密
攔下：暴力破解、帳號枚舉、驗證碼重放
- ▼ ⑦ 完整性驗算：wasmtime 重算分數(見 6.2)
攔下：竄改分數、偽造戰績、補寫回放紀錄
- ▼ ⑧ PostgreSQL 約束：UNIQUE/CHECK、列鎖串行化
攔下：併發競態、重複提交、繞過前七道的缺陷

可信的持久化資料

… 並行運作：稽核日誌以獨立交易寫入；
啟動防呆：關鍵組態錯誤即拒絕啟動 …

圖中各防線的設計考量，在以下兩段中展開說明。

由於 Math Defense 是一套會佈署到網路上、帶有帳號與排行榜的系統，撞庫、跨站請求偽造、權杖竊取等都是真實存在的威脅，因此認證這一塊採取了多層防護。每一張短效的存取權杖內嵌了簽發當下的密碼版本號，每次請求都會比對權杖裡的版本與資料庫現值，一旦玩家改了密碼，資料庫的版本號加一，所有舊的存取權杖就因版本對不上而

立即失效；至於長效的更新權杖則採另一套機制管理，它以隨機值產生、資料庫只保存其雜湊而非明文，每次使用都旋轉換新，偵測到被偷用重放時會撤銷該使用者全部的更新權杖，而改密碼時也會另外顯式地把這些更新權杖一併撤銷。針對暴力嘗試密碼，系統採分級鎖定，連續失敗會被鎖定且鎖定時間逐次拉長，這個狀態存在資料庫裡，因此跨重啟、跨機器都一致。對於已登入狀態下會改變狀態的請求，採用雙重提交的 Cookie 機制防範跨站請求偽造，並以常數時間比對來避免從反應快慢推測內容。多因素驗證的密鑰在存入資料庫前以對稱加密處理，屬於靜態加密，且若加密金鑰未正確設定，後端會在啟動時拒絕運行，避免帶著錯誤組態上線。

除了上述的權杖與鎖定機制，系統在密碼處理與佈署層面還有多項成本低而收益高的防護。密碼以 `bcrypt` 雜湊並明確指定其運算成本參數，且在雜湊之前先擋下超過長度上限的輸入，避免某些實作會把過長密碼默默截斷，進而造成「正確密碼後面亂加字元也能通過」的漏洞；註冊時以密碼強度檢測把關，而這道較耗時的檢查刻意排在限流之後，避免被當成消耗伺服器資源的攻擊面。權杖的驗證會鎖定簽章演算法並核對發行者與受眾，以防演算法混淆類的攻擊；登出的權杖進入黑名單，由背景任務定期清理過期項；登入失敗時則以固定成本的假雜湊比對把回應時間拉平，避免攻擊者從快慢推斷某個帳號是否存在。在佈署層面，生產環境的容器一律移除多餘的 `Linux` 權限並禁止程式事後提權，其中無狀態的前後端容器另把根檔案系統設為唯讀（資料庫與備份容器因需寫入資料卷而不適用）；資料庫完全不對主機開放連接埠而只走內部網路，並支援在執行期改用僅具資料操作權限的低權限角色連線；前端則加上內容安全政策與強制加密傳輸等安全標頭。這些都不是為了炫技，而是一個真的要上線、有帳號與排行榜的系統應有的基本防線。

6.6 前端引擎、框架解耦與工程規範

前端的遊戲邏輯，包括移動、戰鬥、計分與波次，被寫成與畫面完全無關的純 `TypeScript`，引擎本身並不知道 `Vue` 的存在，`Vue` 僅藉由一個橋接用的組合式函式把引擎接上畫面，負責把引擎算出的狀態畫到 `Canvas` 上。引擎採用實體、元件、系統分離的 `ECS` 風格，主迴圈以固定的時間步長推進，搭配可設定種子的偽隨機數；專門的決定論測試以零容差的精確相等，鎖定了相同種子下偽隨機數的抽樣序列逐位元一致，以及若干關鍵呼叫點的結果一致。至於完整一局的重放重建，驗收標準是重算分數與原始紀錄相差在極小的容差內，而非要求整場模擬逐位元相同。把引擎與框架解耦帶來三個好處：引擎可以在沒有瀏覽器的環境中執行、可以自動化地反覆測試、也才能錄製與回放，而最後這一點正是第 6.2 節伺服器端重算防作弊的前提。錄製與回放具體由三個元件支撐，一個把玩家決策連同發生時間記錄下來的錄製器、一個在回放時把這些決策在相同的遊戲時間重新注入的播放器，以及一個讓他人能即時觀戰的用戶端；而引擎之所以能與畫面乾淨分離，靠的是一層投影機制，把可變的引擎物件轉換成不含內部參照的純資料快照供渲染與狀態管理使用，渲染端因此從不直接碰觸引擎內部。

為求多人協作下維持架構不腐化，本專案在持續整合中設置了多道靜態檢查，每次提交都會自動執行。其中包括確認編好的 `WebAssembly` 真的能載入且關鍵函式都在、`TypeScript` 與 `Vue` 的型別檢查、分層引用邊界檢查、事件訂閱與註冊表一致性檢查、註解一律英文的團隊規範，以及在回放相關範圍內禁用會破壞決定論的非確定性數學函式。此外，`C` 的匯出函式清單在建置時會自動生成一份對應的 `TypeScript` 型別宣告，因此若 `C` 端改了函式名稱卻忘了同步，型別檢查會在編譯期就報錯，而非等到執行期才崩潰。另需聲明自動化的邊界：完整的分層依賴規則是寫在團隊規範文件裡的約定，由腳本自動強制的是其中最關鍵的子集，其餘則仍倚

賴人工審查把關。後端另有相依套件的漏洞稽核、資料庫遷移從零重放，以及測試等檢查關卡。

6.7 隱形評量與適性難度推薦

後端內含一個以貝氏統計為基礎的能力評量模型，屬於教育測量領域所稱的隱形評量(*stealth assessment*)，意指在不中斷遊戲的情況下，從遊玩行為本身蒐集能力證據。貝氏統計的精神是先有一個信念，看到證據就更新信念。本專案對玩家的每一項數學技能各維持一個 *Beta* 分布，用來表示對該項能力掌握程度的信念，每當遊戲過程中出現一筆表現證據，就依權重更新這個分布，而後驗的平均值就拿來當作能力估計值，並附上以常態近似算出的可信區間。哪一類事件影響哪幾項技能、各占多少權重，由一張對照表定義，涵蓋七項技能。這個估計值會實際驅動功能：它一方面用來推薦難度：依玩家七項技能後驗的平均值，對應到一個建議的星級(新玩家從第三星起跳)，且只作非強制的提示而不強制鎖關。這個設計的用意是把挑戰維持在「有點難但構得到」的可及範圍，呼應「近側發展區」的概念，並刻意採取建議而非強迫的姿態；另一方面，這個估計值也用來在天賦樹上突顯玩家最弱的技能，並提供給教師端的儀表板。

這套評量的整體骨架是生產級的、在線上運作、也有單元測試覆蓋。同時需要誠實界定它目前的範圍：現階段線上的證據來源只有場次結束時的成就解鎖一種，屬於成功訊號，因此後驗目前是單調上升的。真正帶有「數學答對或答錯」訊號的診斷探針，例如初始作答是否正確、蒙提霍爾是否換對門、極限與連鎖律是否答對，雖然已經在對照表中定義妥當，但把這些遊戲事件實際接上去送出，是計畫中的下一步。架構已經為此鋪好了路，接上之後，模型就能不只往上估計，也能偵測玩家在特定主題上的卡關。為了日後能驗證這套隱形評量到底準不準，後端另建有一套隨機對照試驗的骨架(前測、後測與延遲測的題庫，動機與焦慮量表，以及可在數週後重現的分組方式)；它是為未來研究預留的基

礎，尚未實際施測，不列為已完成的成果。總結而言，這個子系統刻意把測量理論與程式碼綁在一起：從 *Q-matrix*、能力的證據中心設計到難度推薦的自主性原則，多處都在原始碼中直接引用了相關的測量與教育理論文獻，而非僅停留在投影片上的口號。

6.8 班級競賽、領土賽與即時觀戰

除了單人遊玩，本專案另提供以班級為單位的競爭機制。教師可以建立限時的搶領土活動，學生在活動期間以自己的分數爭奪地圖上的格子，並在地區、班級與全球三種排行榜上互相比較；每個活動至多五十格，每位學生能佔的格數另有教師可設定的上限。這層競爭的勝負，仍由前面所述、可被伺服器重新驗算的分數決定，而非單純的答題正確率，因此搶地一樣建立在可信的計分之上。

這類同時湧入的操作對資料一致性是個考驗。當多名學生在同一瞬間搶同一格時，後端將以 *PostgreSQL* 的諮詢鎖把同一名學生的並行請求序列化，再加上第 6.4 節提到的「每格至多一人佔據、每場遊戲至多用於一次搶格」這兩道資料庫約束，從根本上杜絕同一格被重複佔據、或以分身同時灌多格的可能。活動到期後的結算與排名計算，則交由一個背景排程定期處理，與任何單一請求解耦。

即時觀戰以行程內的 *WebSocket* 廣播實作。每一觀眾各有一個容量有限的訊息佇列，當某觀眾接收過慢時，系統將丟棄給該觀眾的更新，不讓全廣播被它拖住；觀眾的權杖也會定期重新驗證，一失效就在連線中斷開，避免已被撤銷的人繼續觀看。

6.9 測試、壓力測試與效能

品質保證方面，後端約有三十多個測試檔，涵蓋領域聚合、路由、領域不變式、登入鎖定、權杖黑名單、跨站請求偽造、計分對拍、回放重算、*wasmtime* 執行環境等，以一個真實的 *PostgreSQL* 測試資料庫執行，行覆蓋率約為百分之八十一。前端約有九十個測試檔，涵蓋各系統、引擎、*WebAssembly* 與 *JavaScript* 的數值對拍、回放決定論、鍵盤放置塔等，以 *Vitest* 搭配 *happy-dom* 執行，

行覆蓋率約為百分之五十七；前端覆蓋率較低的部分多集中在畫面元件，而核心的遊戲引擎與計分則有決定論測試與數值對拍這類高價值測試在守。需要說明的是，這些覆蓋率數字是手動量測的一次性快照，持續整合本身並未設定覆蓋率門檻。

效能與負載方面，專案另有獨立的壓力測試。以 k6 模擬約一百名使用者同時操作的尖峰情境，跑完整的「建立場次、多次更新、結束、查排行榜」流程，檢查通過率為百分之百、錯誤率為零；不過這項量測是在與後端同一台機器上進行的，因此延遲數字屬於樂觀的下限，尚未測到真正的崩潰點。至於遊戲最吃效能的計算熱路徑，在三十二座塔對三百個敵人這種極端規模下，單幀計算的百分之九十五分位數約為一點二四毫秒，遠低於每秒六十幀所對應的每幀十六點六七毫秒預算；這個數字是純計算的基準，不含畫面渲染，用來確認演算法本身在最壞規模下仍有充裕餘裕。

7. 創意、新意與原創性

本專案的創新主要展現在設計理念上。最根本的一點是把數學變成機制本身，而非進入遊戲的門檻。在多數數學練習應用程式裡，玩家是先答對題目、再獲得一段與數學無關的遊戲獎勵，數學與樂趣始終是分離的。Math Defense 則讓數學運算直接成為攻擊：向量內積決定矩陣塔的傷害、對多項式做微分或積分的結果決定微積分塔生成的寵物、極限的答案決定極限塔的效果。玩家是在玩數學，而不是在數學與遊戲之間切換，這是與一般數學練習軟體最本質的差異。

第二個創新是把抽象的數學概念視覺化、遊戲化。整個戰場就是一個座標平面，玩家要防守的據點被定義為「所有路徑曲線的唯一共同交點」，敵人沿著程式自動生成、且都通過該點的多項式曲線前進。函數圖形、曲線交點這些原本停留在課本上的抽象物件，在這裡變成了看得見、要與之互動的遊戲元素。由此延伸出全遊戲最獨特的開場設計：初始作答。玩家在每一關開始之前，先親手解一組

聯立方程式，算出來的交點正是自己接下來要防守的據點，解題在這裡不是附加的測驗，而是一場偵察；願意承擔風險的玩家可以選擇盲打，在完全不顯示敵人路徑的情況下作戰，把對函數圖形的掌握化為真正的實力考驗。

第三個創新是把機率課悄悄藏進遊戲。蒙提霍爾事件刻意不揭露「換門比較有利」這個違反直覺的數學事實，而是讓玩家從反覆遊玩、累積成敗經驗的過程中自行體會。這是一種在玩家不自覺的情況下進行的教學設計，把抽象的條件機率變成一個玩家親身做選擇、親眼看到結果的決定。

第四個層面是技術與工程的嚴謹度本身就構成一種創新。用 C 與 WebAssembly 達成跨執行環境的位元一致，使得伺服器能以同一份邏輯重算分數來防止作弊，把教育遊戲常被忽略的「分數可信度」問題謹慎地解決掉。最後，成就、天賦樹、班級領土競賽與排行榜把單局遊玩銜接成一條長期的學習進程，後端的能力估計模型已實際用於星級難度建議，並為日後的個人化出題與動態難度調整預留了基礎。關於在教育理論上的設計依據，另見專案內的 docs/Educational_Theory_Analysis.md。

8. 美術與介面設計

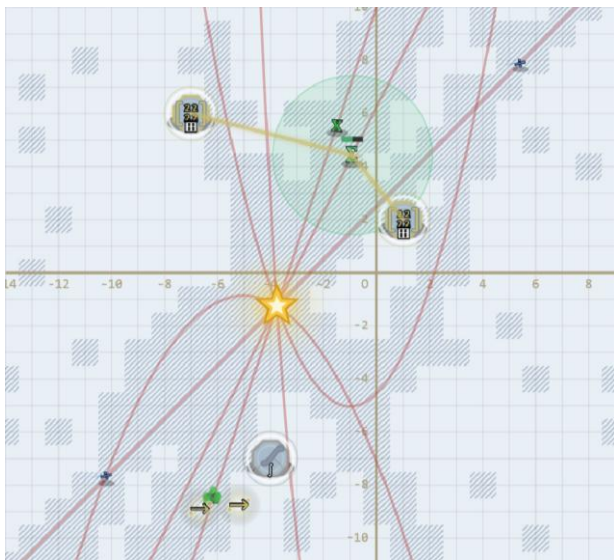
8.1 視覺風格與數學符號美術語言

Math Defense 以座標平面作為主視覺，戰場就是一張數學圖紙，把「數學即機制」的理念一路延伸到「數學即美術」。在畫面上，看不到一般塔防常見的怪物立繪或現成塔圖，場上的每一個物件都是即時以程式繪製出來的數學符號或數學儀器。

敵人不是怪獸，而是會動的數學符號：一般兵是一個走起來會微幌的 x ，快速兵是傾斜並拖著兩道殘影的除號，強壯兵是一團圍著等號糾纏的 $(+--)$ ，分裂兵是一個分數、死亡時會沿著分數線裂成兩半，輔助兵是一個會隨光環呼吸的 Σ ，再生兵是一個被旋轉虛線環包住的 $\lim$ ，堡壘兵是兩道打了鉚釘的平行牆，蟲群是一個帶著三個小 ε 衛星繞

行的 $\varepsilon \cdot A$ 型頭目是一個無解方程式般的全稱量詞 $\forall$ 、外圍漂著閃爍的方程式碎片，B 型頭目則是一條雙紐線(8 字形的悖論迴圈)，周圍繞著循環箭頭。

七種塔則是各自對應其數學主題的儀器：魔法塔是一張畫著會起伏的正弦曲線的羊皮卷軸，掃描雷達塔是一具在刻度弧上掃動的六分儀，速射雷達塔是兩圈反向旋轉的星盤，狙擊雷達塔是一架架在三腳架上、會追著最近敵人轉的黃銅望遠鏡，矩陣塔是一對漂浮的二乘二中括號、格內還有滾動的數字，極限塔是一個沿著兩條虛線漸近線不斷上升、卻永遠構不到上界的點，微積分塔則是一個緩慢旋轉的 $\int$ 符號，開火時持續噴出 dx 、 dy 字樣的粒子。色差描邊則被當成區分敵我的視覺語彙：敵方符號帶有青色與洋紅錯位的描邊，刻意營造出彷彿數學世界出現故障訊號的意象(glitch 美學)；我方的寵物改用純青色描邊、法術特效改用金色描邊，讓玩家在高速戰鬥中能憑色彩立即分辨敵我。這套美術讓玩家在戰鬥中看到的每一個元素，本身就是一則數學提示，而不只是裝飾。



8.2 設計系統與視覺識別

在介面層，本專案建立了一套以去飽和的莫蘭迪(Morandi)色系為基礎的設計系統，把所有顏色集中定義為設計變數(design token，以 CSS 變數實作)，作為前端唯一的調色來源，確保全站數十個畫面共用同一套色彩語言。七種塔各有專屬的識別色，

而魔法塔還會依減益或增益模式在紫色與淡紫之間切換，讓玩家一眼就能判讀它當下的狀態。數字與分數類的讀數採用等寬字體以利對齊，初始作答、連鎖律題目與原理說明等處的算式，則以專業的數學排版函式庫 KaTeX 渲染成清楚的數學式，而非難讀的純文字符號。整體配色與 Logo 維持一致的識別，主選單與登入頁的 Logo 是一圈會緩慢旋轉、四個方位點綴幾何符號的金色光環，並會在玩家開啟「減少動態效果」時自動停止旋轉。

這套美術在工程上也照顧到跨平台與不同螢幕。許多數學符號在非 Windows 系統の後備字體裡並不存在，因此像平行線與循環箭頭這類高風險字元，並非直接印出文字，而是改以向量路徑手工繪製，使遊戲在任何作業系統上看起來都一致，堡壘兵的平行牆與 B 型頭目的雙紐線都是這樣畫出來的。遊戲畫面本身固定以一千兩百八十乘七百二十像素的座標系繪製，再透過單一的 CSS 縮放搭配視窗尺寸監看，等比例放大或縮小以填滿不同大小的螢幕，並設有放大上限以免畫面過度模糊；縮放過程中座標運算、滑鼠對位與高解析度顯示都維持不變，只有視覺外框被縮放。



Tower's Colors

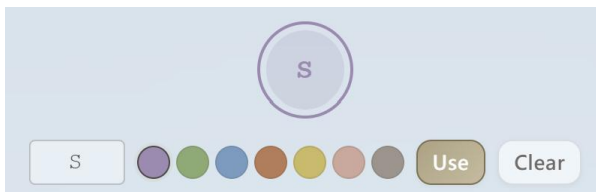
8.3 個人化與外觀客製

玩家可以客製自己要防守的核心據點(也就是所有曲線的共同交點 P^*)外觀，這是一個容易被忽略的個人化亮點。標記樣式提供星星、猩猩與自行上傳的自訂圖片三種，其中自訂圖片會在前端先

縮放成二百五十六乘二百五十六並壓在三 MB 以內，後端再以檔頭位元組與影像尺寸加以驗證，以防解壓縮炸彈；據點被敵人觸及時的受擊特效則可在碎裂、哭泣、生氣或隨機之間選擇，而「隨機」是透過遊戲本身的偽隨機數決定，因此回放時仍能逐次重現完全相同的特效。除了據點，玩家也能設定個人頭像，以一到兩個字母搭配取自七種塔識別色的底色組成，並儲存在伺服器端。這些客製選項都在結構描述、領域聚合與資料庫約束三層各驗證一次，確保沒有可以繞過的途徑。



函數核心之自選標記樣式 & 受擊特效(上圖)



設定玩家個人頭像(上圖)

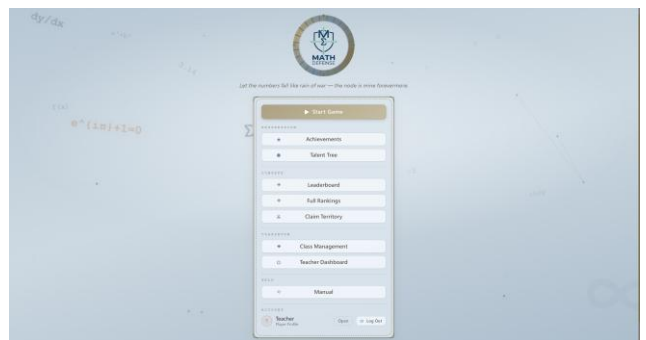
8.4 無障礙設計

本專案在介面設計上亦著力於無障礙，這也是設計面的一個亮點。每座塔除了用顏色區分之外，都另外配有一個獨有的幾何符號(◆、●、●、●、田、∞、J)，因此即使是色覺辨識有困難的玩家，也能單憑符號區分塔的種類，符合無障礙準則中「不單靠顏色傳達資訊」的要求。畫面設有一個即時播報區，會以輔助技術可讀的方式宣告階段的轉換與生命值警告。系統尊重作業系統之減少動態效果的偏好設定，在玩家開啟該設定時降低環境動畫的強度。塔的放置支援完整的鍵盤操作，玩家可以只用方向鍵與 Enter 完成佈陣，符合「所有功能皆可由鍵盤操作」的準則，而鍵盤選取游標另以深色外暈加上亮色內環呈現，確保在任何底色上都有足夠的對比。可點擊的按鈕都保有四十四乘四十四像素以上的點按範圍，彈出視窗則會把鍵盤焦點限制

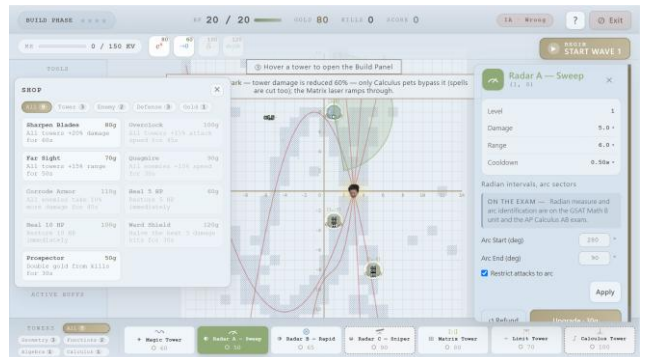
在窗內，並在關閉後把焦點歸還，避免焦點逸失。此外，敵人路徑的標籤會依玩家近期初始作答的正確率淡入淡出，讓系統能不打斷遊戲的前提下，對掌握程度不同的玩家給予不同程度的視覺提示。

8.5 遊戲內手冊與實機畫面

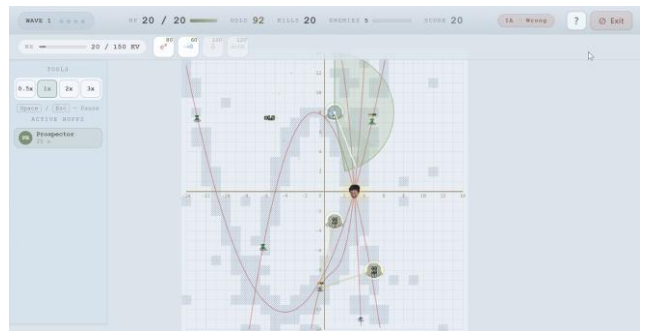
為了降低上手門檻，遊戲內另附有操作手冊，透過遊戲中的手冊視窗呈現玩法與各塔、各敵人的詳細說明，玩家不必離開遊戲就能查閱。以下另預留數張實機截圖佔位，分別呈現主選單、建造階段、進攻波次與結算畫面，以具體展示介面與美術。



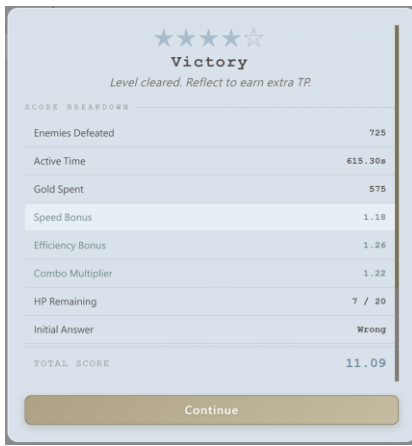
首頁主選單(上圖)



遊戲進行中 - 建造階段(上圖)



遊戲進行中 - 進攻波次(上圖)



★★★★★	
Victory	
Level cleared. Reflect to earn extra TP	
SCORE BREAKDOWN	
Enemies Defeated	725
Active Time	615.30s
Gold Spent	575
Speed Bonus	1.18
Efficiency Bonus	1.26
Combo Multiplier	1.22
HP Remaining	7 / 20
Initial Answer	Wrong
TOTAL SCORE	11.09
Continue	

結算畫面(左圖)

9. 安裝與執行方式(Ubuntu 24.04)

完整執行步驟詳見專案中所提供的 README.md 與 UBUNTU_SETUP.md，以下為摘要，目的是確保能順利執行。整個專案在 Ubuntu 24.04 環境下可以運作，需要留意的是版本落差:Ubuntu 24.04 的 apt 預設提供的 Node.js 為 18、Python 為 3.12，都低於本專案所需的 Node.js 26 以上與 Python 3.13 以上，因此採用手動安裝路徑時必須另行升級；而本專案使用的 PostgreSQL 16 則可直接由 apt 取得。若採用建議的 Docker 路徑，上述版本問題完全由容器映像處理，無須在主機上手動升級。

Emscripten SDK 僅在需重新編譯 C 數學核心時才用得到，而由於編譯產物已隨專案提交，且數學層另有純 TypeScript 後備實作，一般情況下無須重建即可遊玩。需注意，專案內隨附的 emsdk 是 Windows 版本，在 Linux 上無法直接產生可用的編譯器，因此 Linux 上若真要重編，可改用 Docker 重建後端映像，或另行安裝原生的 Emscripten。

建議的執行方式是使用 Docker。安裝 Docker 與 Compose v2 後，取得程式碼，先複製環境變數範本，也就是執行 `cp .env.example .env`，接著編輯這份 .env，至少要設定一個長度三十二字元以上的 SECRET_KEY、把 DATABASE_URL 與 POSTGRES_PASSWORD 裡的預設密碼換掉，並設定一把用於加密多因素驗證密鑰的 TOTP_ENCRYPTION_KEY；後端在這幾項未設妥之前會拒絕啟動，這是刻意的安全防呆。設定完成後

執行 `docker compose up` 即可，系統會依序啟動 PostgreSQL、後端與前端三個服務，並在後端首次啟動時自動套用資料庫遷移。前端會在 5173 埠、後端 API 在 8000 埠、PostgreSQL 在 5432 埠提供服務，健康檢查端點為 8000 埠的 /health。這三個埠在開發環境都只綁定於本機，不對區域網路開放。

若不使用 Docker 而要手動執行，則需先備妥一個運作中的 PostgreSQL 16 以及對應的資料庫與角色，具體指令見 backend/README.md。後端的部分，進入 backend 目錄建立虛擬環境並安裝相依套件，複製一份 .env 並把資料庫主機由 postgres 改為 localhost，再以 uvicorn 啟動；資料庫遷移會在後端首次啟動時自動套用。前端的部分，進入 frontend 目錄執行 `npm install` 安裝相依套件後，以 `npm run dev` 啟動開發伺服器即可。

最後提醒一個容易誤判的情況，以利順利執行。本專案的認證 Cookie 預設帶有僅限安全連線的屬性，因此若在遠端或無頭主機上執行，卻直接以該主機的區域網路位址(而非 localhost)從另一台電腦開啟頁面，瀏覽器會在純 HTTP 下丟棄這個 Cookie，造成登入「沒有任何錯誤訊息卻一直失敗」的假象，而非程式本身有問題。遇到這種情形，只需透過 SSH 通道把連接埠轉送到本機，讓瀏覽器仍以 localhost 的身分連入即可正常登入。

10. 開發歷程與遭遇的困難

本專案於五月十二日進行提案、六月九日進行報告、六月十二日繳交。開發過程歷經數次較大的重構，從第一版到第三版逐步演進，版本紀錄可以佐證。第二版把早期較雜的塔種整併為現在這七種以數學概念為核心的塔，使每一座塔都明確對應一個數學主題；第三版則把計分公式升級為以擊殺價值為基礎的核心再乘上難度乘數的形式。

開發中遭遇的最具代表性的困難，是跨執行環境的浮點決定論。最初前端顯示的分數與後端重算的結果，會在小數的最末幾位出現分歧。這種誤差無法靠重試消除，因為它是系統性的:同一行 C 程

式碼經過不同的編譯與執行環境，可能走上不同的運算路徑。追查後確認分歧有兩個來源，其一是編譯器會對浮點運算式做代數重寫，例如把連乘加融合成單一的乘加指令，造成末位差異；其二是 `sin`、`log` 這類超越函數若借用執行環境內建的實作，不同引擎之間本來就不保證逐位元一致。解法因此也分兩路：以編譯旗標禁止任何會改變捨入結果的重寫，並把 `musl` 標準函式庫直接編進 `WebAssembly`，讓超越函數隨二進位檔附帶；最後以三方對拍資料把結果鎖住，日後任何再度分歧都會被測試攔下。這個過程也帶出一個工程教訓：由於編譯後的 `WebAssembly` 匯出會優先於 `JavaScript` 與 `Python` 的後備實作被採用，任何對計分公式的修改都必須連帶重新編譯數學核心並重新產生對拍資料，否則新公式不會真正生效。

另一個困難是 `WebAssembly` 的記憶體管理。把陣列傳給 C 需要手動配置與釋放記憶體，稍有不慎就會洩漏。解法是在 `WasmBridge` 中以一個會自動釋放的包裝函式，管理那些需要頻繁配置與釋放的浮點緩衝區的生命週期，確保即使中途拋出例外也不會留下未釋放的記憶體。

11. 分工

本組依各自專長分工，每位組員負責明確的模組，跨模組的整合則透過 `Pull Request` 審查流程共同把關。另須聲明，如 2.2 節所揭露，程式碼的撰寫過程有相當比重由 LLM 依規格實作；因此以下所稱某組員「負責」或「完成」某模組，指的是該模組的設計、規格、實作監督與品質驗收由其主導，而非每一行程式皆由其親手輸入。

黃晨瑋(Full-Stack Developer · Tech Lead · Game Designer)負責系統架構、數學核心與後端開發，專案的工程骨幹由其擬定與實作。具體工作包括第五章所述三層架構的整體設計，含前端 ECS 風格引擎、固定幀率主迴圈，以及管理 `WebAssembly` 記憶體生命週期的 `WasmBridge`；第六章與第十章所述的數學核心亦由其主導完成，以 C 語言編譯為

`WebAssembly`，透過編譯旗標與內嵌 `musl` 標準函式庫達成核心數學函式跨執行環境逐位元一致的決定論，再以三方對拍資料把結果鎖進測試；以擊殺價值為基礎的計分公式、伺服器端的分數重算驗證與確定性回放等防作弊機制同樣由其設計與把關。後端方面，由其建立領域驅動的分層架構、安全防護與資料庫設計。玩法上，七種數學主題的塔與十種敵人的機制設計皆由其主導；整合測試階段，並依品質保證流程建立的缺陷追蹤紀錄，完成橫跨多模組的錯誤修復。

王晨媛(Frontend Developer · Design Lead · QA)負責前端應用、設計系統與品質保證，玩家在遊戲中所見的介面層多出自其手。具體工作包括 `Vue SPA` 各主要介面的實作，以及盤面、塔與法術區域等遊戲畫面的渲染與視覺調校；第八章所述的設計系統亦由其建立，包括以設計變數集中管理的莫蘭迪色系調色、七種塔的識別色、等寬讀數與 `KaTeX` 數學排版等元件規範，使全站畫面共用同一套視覺語言；同章的無障礙設計亦由其落實，涵蓋塔放置的完整鍵盤操作、輔助技術可讀的即時播報、對減少動態效果偏好的尊重，以及點按範圍與焦點管理等細節。個人檔案與自訂頭像功能從介面到伺服器端持久化的實作亦由其完成。品質保證方面，由其主導逐功能的系統性測試，建立貫穿整合階段的缺陷追蹤紀錄，並推動從發現、修復到驗收的品質迴圈，整合測試階段橫跨各模組的錯誤修復皆以這份紀錄為依據，使其成為整個專案品質的最後一道把關。本報告初稿亦由其統整資訊撰寫而成。

邵顯智(Visual Developer · Game Artist · Video Editing)負責遊戲美術與媒體製作。具體工作包括第八章所述以數學符號與數學儀器構成的塔與敵人視覺，並以色差描邊建立出區分敵我的 `glitch` 視覺語彙，敵方用青色與洋紅錯位、寵物用純青、法術用金色；四種法術施放時的特效及其圖示亦出自其手。美術之外，其自開發早期即持續試玩並回饋操作手感，遊戲操作介面的多處細部調整即源自

這些建議；第十三章所附的專案介紹影片，亦由其獨力剪輯與製作完成。

12. 結論與未來展望

Math Defense 以「數學即遊戲機制」為核心，示範了數學概念可以是遊戲樂趣的來源，而不是阻擋玩家的門檻。在技術上，本專案以三層架構，在不依賴任何現成遊戲引擎的前提下，從零落實了一個可遊玩、可競賽，並且具備伺服器端防作弊與可驗證決定論的完整系統，同時把無障礙設計納入考量。整個專案最具分量的成果，是讓同一份以 C 語言撰寫的數學邏輯，既能在瀏覽器中驅動遊戲，又能在伺服器上重算驗證分數：核心數學函式的輸出跨執行環境逐位元一致，整局分數的重算則以極小容差驗收，使一款教育遊戲的分數具備可信度。

未來展望方面，首要的一步是把已在評量模型中定義妥當的診斷探針實際接上遊戲事件，讓能力估計能依玩家在各數學主題上的答對與答錯動態調整，進而支撐真正的個人化出題與難度自適應。其餘可能的延伸方向包括擴充更多對應數學主題的塔，例如機率分佈與線性代數，加入多人即時對戰，以及把 C 核心進一步抽出，讓它也能在原生編譯環境下接受一致性對拍驗證。

13. 參考資料與素材來源

專案內的文件方面，系統與設計的詳細說明散見於

- README.md
- ARCHITECTURE.md
- DATABASE_SCHEMA.md
- SECURITY.md
- UBUNTU_SETUP.md
- Math_Defense_Spec.md、wasm/README.md
- docs/Educational_Theory_Analysis.md

本專案另有錄製一部介紹影片，其連結如下 <https://youtu.be/7oKve4Sg1x8>。

主要的開源相依套件，前端包括 Vue、Pinia、Vue Router、Vite 與 Vitest，後端包括 FastAPI、Uvicorn、SQLAlchemy、Pydantic、PyJWT、bcrypt、slowapi、wasmtime 與用於加密的相關函式庫，

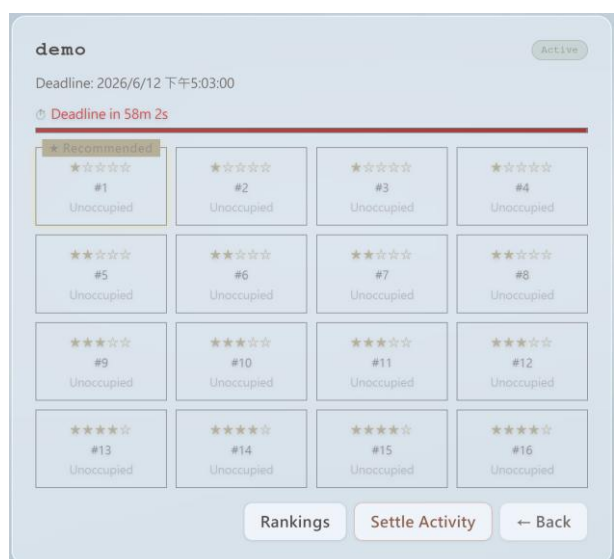
WebAssembly 工具鏈則為 Emscripten 與 musl 標準函式庫，皆依各自的開源授權使用。演算法方面，偽隨機數產生器參考了 M. E. O'Neill 提出的 PCG 系列演算法，屬公眾領域。如第二章所述，班級領土競賽的玩法概念參考了葉丙成教授團隊的 PaGamO。

影像、字型與音效方面，Logo 與遊戲的視覺美術皆為本組自製，並未另行取用第三方的美術素材；遊戲音效則全部以程式化方式自行合成，由基本波形產生、釋為 CC0，不含任何第三方音訊。至於專案所使用的第三方函式庫及其附帶資產，例如數學排版函式庫 KaTeX 隨附的字型，則依各自的開源授權使用，詳見專案的 NOTICE 檔。

14. 附錄



排行榜(上圖)



領地爭奪(上圖)